

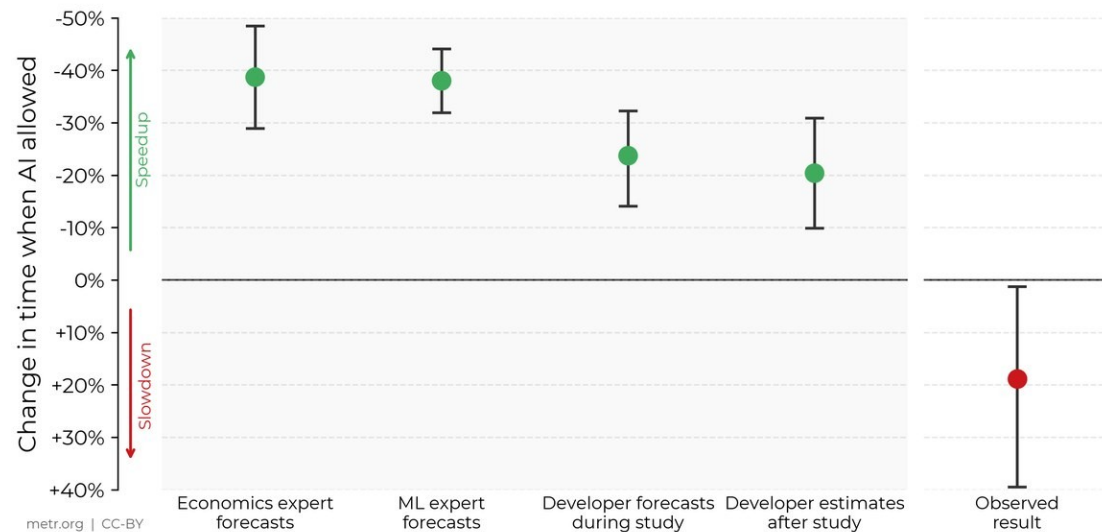
16 experts were sure AI sped them up — and got the sign wrong



Against Expert Forecasts and Developer Self-Reports, Early-2025 AI Slows Down Experienced Open-Source Developers



In this RCT, 16 developers with moderate AI experience complete 246 tasks in large and complex projects on which they have an average of 5 years of prior experience.



Predicted -24% · believed -20% · actual +19% time

METR experiment, first half of 2025

16 experienced open-source developers · 246 real tasks in their own familiar repositories; measured real time, not perception

Three numbers about one and the same thing

Predicted before the experiment: AI will speed up by -24%

Believed after working: sped up by roughly -20%

Measured actual: with AI, tasks took +19% longer

— METR RCT, n=16 experienced OSS developers, 246 tasks, 2025

The professionals got the magnitude right but the sign wrong. "It feels like the tool helps" is a hypothesis, not data; reliability comes from discipline with built-in verification, not from perception. ^[1]

[1] METR — RCT: AI and experienced OSS developers (+19% time)

04

LECTURE 4

AI across the software development lifecycle

Course "Deliberate use of AI" · software developers



Reliability comes not from the tool but from engineering discipline by phase: which artifact and which check each one needs.

0
Intro

1
Requirements

2
Architecture

3
Implementation

4
Testing

5
Review+Sec.

6
Delivery+

7
Synthesis

We stand on the foundation of Module 1 — take one industry and break it down by phase



From Lecture 1

The layered picture "model → chat → agent → application"; a prompt as "role + task + context". We use it as a vocabulary.



From Lecture 2

Why AI produces "almost right" text: the answer is generated token by token, so plausible ≠ correct. In one phrase.



From Lecture 3

The complexity ladder ("stay on the lowest rung") + the criterion "when not AI at all" — a deterministic task is solved by ordinary code.



From Lecture 3

The agent loop plan → act → check → iterate (4 points of failure) + prompt injection (defense is architectural, not by filtering).

We carry this over intact and don't re-explain it — we only apply it to software development. Module 1 gave the apparatus for choosing an architecture; Lecture 4 shows which discipline makes AI in one industry reliable by phase — and where it breaks without it.

The lecture's question is about discipline, not about the tool

AI writes code ever better — but what makes AI development reliable?

Not the tool, but engineering discipline by phase: which human-owned artifact and which check each phase needs, where the methods of different players converge — and what is not delegated to the tool in any phase. ^[1]

Which tool is best?

Goes stale in a quarter; "best" can't be named without "for what, in which phase, in which mode".



Which practice is warranted in the phase?

Which artifact does it end in? Where is a human mandatory? — stable for years.

The tool is secondary, the practice primary. When answering, an engineer names not a logo but the phase's appropriate practice, its artifact, and the point of mandatory human control — knowledge that will outlive the swap of any vendor. ^[2]

This lecture is a digest of practices: modern approaches of the leaders + time-tested classics



A. Modern practices (leaders)

[1] [Anthropic — Claude Code / AI-Native SDLC](#)

agentic git loop: each stage commits an artifact; human at the gates

[2] [OpenAI — Model Spec](#)

spec-as-contract; clause = example prompt = test

[3] [GitHub — Spec Kit](#)

"intent is the source of truth"; small verifiable tasks

[4] [Google — DORA 2025](#)

"AI amplifies what is already there"; seven capabilities

[5] [Thoughtworks — Exploring Gen AI](#)

the assistant suggests, the developer owns; harness engineering

[6] [Willison — Vibe engineering](#)

the disciplines an LLM rewards: tests, plans, reviews



B. Time-tested classics

[7] [Brooks — No Silver Bullet](#)

essential vs accidental complexity; "what to build" is the human's

[8] [Beck — TDD: By Example](#)

red-green-refactor; test-as-specification

[9] [Nygard — ADR](#)

immutable "why" records under version control

[10] [Ford/Parsons — Evolutionary Architectures](#)

fitness functions: "fitness" is objective and automatic

[11] [Fowler — Refactoring](#)

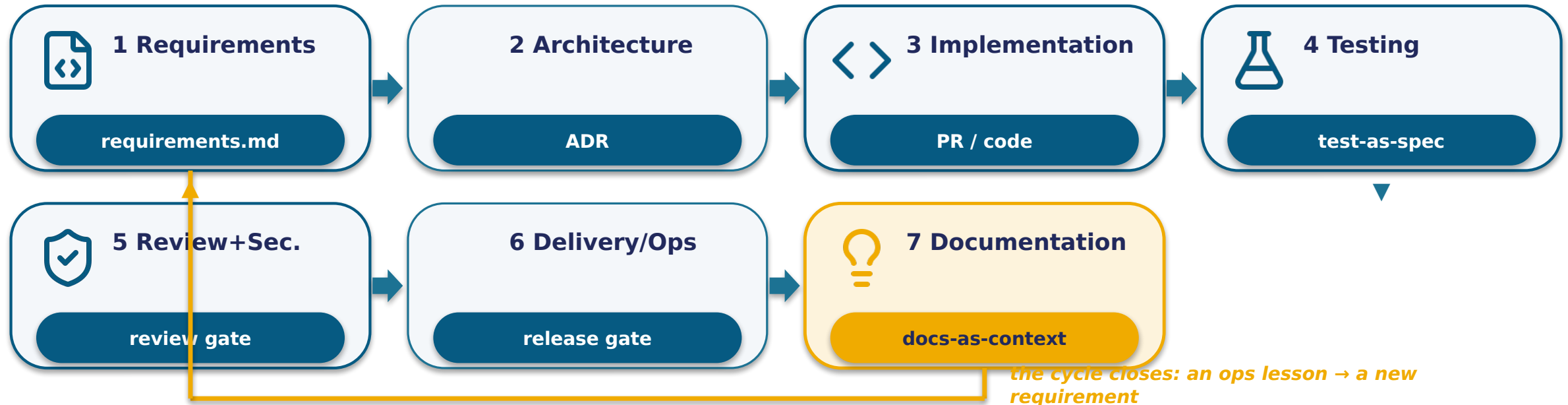
the discipline of small verifiable changes

[12] [Brown — C4](#)

architecture-as-code: a textual, diffable model for AI

**Tools will change over quarters — these practices are stable: they rest on the nature of complexity, not on product maturity.
All names are clickable links to the primary sources.**

Discipline is a cycle of phases, and each phase is owned by a human through its artifact



A phase is a stage with its own input / output / artifact.

AI works INSIDE the nodes (drafts requirements, proposes an ADR, writes code in a PR), but each node is owned by a human — who reads, edits, accepts, is accountable ^[1].

The practices mature unevenly

strong: requirements, implementation, testing, review · thin: architecture, delivery, operations · bright spot: documentation.

Anthropic ^[1], OpenAI ^[2], DORA ^[3], and Thoughtworks ^[4] independently converged on this phase skeleton — so what we have is a method, not a fashion.

Autonomy is a property of the mode, not the brand

The autonomy ladder — a supporting lens



- A autocomplete**
finishes the line; the human accepts each one
- B small tasks**
a function/fix in a dialog; the human reviews after
- C coding agent**
plans, edits files, runs tests; the human does the merge
- D orchestrator**
takes a task from the tracker → PR; the human is the prod gate

Mode $\neq$ brand — one product lives on several rungs at once ^[2]

Copilot — A, B, C and D · Cursor — Tab (A), Cmd-K (B), Composer (C) · Claude Code — C, rising to D

B ↔ C does it iterate and run tests without you? (yes → C)

C ↔ D where the task comes from, where it lands (tracker → PR → D)

The higher the autonomy, the stricter the criterion "a human is mandatory here" and the tighter the harness. "We use Copilot" reports neither the level nor the phase — name the mode and the phase, not the logo. ^[1]

The method decides, the tool executes

A methodological practice is the decision about which artifact to produce, in what order, with what check, and who is accountable. The tool is the executor: interchangeable and secondary.



Volatility

Tools and their "leadership" change over quarters; practices are stable for years — they rest on the nature of complexity (Brooks ^[1]), not on product maturity.



The DORA multiplier

"AI amplifies what is already there" ^[2]: build the discipline first, not pick the tool — a tool without discipline multiplies chaos.



Accountability

The tool is not accountable for consequences, the human is; accountability materializes in artifacts and gates, that is, in the practice.

"We adopted an AI tool" $\neq$ "we adopted an AI discipline". The tool is there, the practice is not — and the DORA multiplier works the wrong way. Behind this stand all the failures of the lecture: prompt-and-pray, poisoned context, the 70% problem, the Replit incident — everywhere the tool applied without the practice.

Method-first order: first the phase's practice (artifact, gate, who is accountable), then the tool to fit it. The tool is chosen to fit the discipline, not the discipline to fit the tool. ^[3]

SECTION 1

Requirements — the first artifact, before any code

The cycle's first phase turns a vague intent into requirements. What leads here is not the tool but the discipline of requirements (spec-driven): between intent and code a human-reviewable artifact is placed, and the decision "what exactly we need" stays human.



0

Intro

1

Requirements

2

Architecture

3

Implementation

4

Testing

5

Review+Sec.

6

Delivery+

7

Synthesis

The first development artifact is requirements, not code



Repository: requirements are versioned next to the code ^[3]



requirements.md (spec.md)

what the system must do



design.md

constraints and decisions



tasks.md

decomposition into tasks



src/...

code — generated from requirements

The order is enforced: requirements → design → tasks (Kiro / Spec-Kit ^[3]). The costliest errors are in this phase.

OpenAI Model Spec ^[2]

living, versioned Markdown: requirements are a durable, reviewable, diffable artifact, not fleeting prompts.

Sean Grove, OpenAI · "The New Code" ^[5]

"the source requirements — that is the valuable artifact"; code is "structured communication".

Martin Fowler ^[4]

the bottleneck of AI development is intent: the model writes code ever better, but expressing what to build is still hard.

Grove's "code ~10-20% of the value" is a rhetorical provocation, not an empirical measurement: its job is to shift the focus to requirements.

AI is strong at structuring and completing intent; the intent itself — what the system needs — is decided by the human ^[1].

How to run requirements: structure (how to write) + process (how to maintain)

✓≡ STRUCTURE — how to write requirements

- **Stories + acceptance criteria** ^[3]

"As a <role>, I want <goal>, so that <benefit>" + verifiable criteria for each story.

- **EARS notation** ^[7]

"WHEN <trigger>, the system SHALL <response>" (Mavin 2009) — removes "should/may", makes a requirement testable.

- **Functional vs non-functional** ^[6]

behavior separate from characteristics (latency / cost / security); NFRs are enforced by fitness functions.

- **requirements → design → tasks** ^[3]

the enforced order of 3 files (Kiro / Spec-Kit); DoD — small independently testable units.

🔄 PROCESS — how to maintain requirements

- **Elicitation: the interrogatory LLM** ^[4]

the model ASKS questions (Fowler's "Interrogatory LLM"), surfacing unstated assumptions — instead of "prompt-and-pray".

- **Review and sign-off BEFORE code** ^[1]

requirements are reviewed and signed off by a human before generation; accept/reject = "the merge".

- **Versioning next to the code** ^[3]

requirements are diffable Markdown in the repository, not in a wiki / chat; a durable artifact, not a fleeting prompt.

- **Syncing with change** ^[9]

keep current like an ADR; the human owns "what to build", AI owns structure and completeness.

Durable pattern: EARS + decomposition + requirements-as-check + human sign-off — will outlive any tool. Hype: "our pipeline of commands = requirements discipline". Tools execute, the method leads.

prompt-and-pray: the bug is not in the code, but in a requirement nobody checked



The visible 'works in the demo' is the tip; below the water are dozens of unstated assumptions the model defaulted on.

prompt-and-pray — one vague prompt ('build me a booking system') and hope. This skips the discipline: no requirements artifact, no human checkpoint between intent and code.

The model silently fills in the decisions: a booking in the past? overlapping bookings? who cancels someone else's? time zones? — for each it takes a plausible default. It 'works' in the demo, breaks on the first real conflict.

The code is correct relative to what the model assumed. The bug is not in the code — it is that nobody checked the assumptions; they are invisible in the code.

The mirror-image extreme (Encarnacao, 'The Emperor's New Code'): 'the spec = the single truth, you need not read the code'. But a spec underspecifies behavior; 'I'll regenerate from the spec' is a new guess, not the same product. The code remains the source of truth.

The bottleneck is not the model's ability to write code, but the precision of stating intent (essential complexity, Brooks ^[1]). The alternative is not 'no AI', but restoring the human checkpoint: requirements accepted before code ^[2].

SECTION 2

Architecture — before code, and it must be managed

After requirements comes not code straight away, but architecture: deciding what to assemble the system from. This is essential complexity, led by the human; the leading practices — ADR, fitness functions, architecture-as-code — teach you to manage it with AI, not delegate it to AI.



0

Intro

1

Requirements

2

Architecture

3

Implementation

4

Testing

5

Review+Sec.

6

Delivery+

7

Synthesis

After requirements comes architecture, not code straight away

what is needed

requirements



what to assemble from

architecture

**cannot
skip**



how to write

code

The phase's output is a small number of hard, hard-to-reverse forks: component boundaries, the data model, the priority of quality attributes.

Jump straight to code →

Architecture erosion — the gap between what was intended and what was implemented, a decay in maintainability.

Codebase cognitive debt (Thoughtworks Radar, Hold ring ^[2]): the gap between how the system is built and the team's understanding — it 'lives in people's heads', not in artifacts. The remedy Radar names — architectural fitness functions ^[3].

'Deciding what to build' is essential complexity (Brooks, 'No Silver Bullet', 1986 ^[1]): a trade-off choice is not delegated. AI is useful only at the periphery — options, explaining a pattern, a draft diagram.

Four practices for managing architecture with AI (tools are secondary)

 ADR	 Fitness function	 C4 / arch-as-code	 Evolutionary arch.
WHAT IT IS Half a page of an immutable record per decision (context·decision·status·consequences); stores the 'why'	Auto-check of an architectural attribute on every commit ('payment does not depend on the UI'; 'response < 200 ms')	Architecture machine-readably (C4: Context/Container/Component/Code; DSL — PlantUML/Mermaid/Structurizr)	ADR + fitness functions + arch-as-code together = incrementality + managed change
WHO PRESCRIBES IT Nygard 2011 ^[1] ; Radar — ADOPT ^[4]	Thoughtworks; Rebecca Parsons ^[4]	Simon Brown (C4) ^[3] ; Structurizr	Ford, Parsons, Kua ^[2]
AI ROLE (SECONDARY) edits, cross-checks — but the human decides and justifies	convenient for writing fitness functions; they also validate generated code	reads as context, generates diagrams; drift detection — Structurizr (model vs code)	executes inside each of the three practices
WHERE THE HUMAN IS the fork's author = the ADR's author	decides which invariant is critical	owns the textual model	holds the direction of evolution

The durable pattern: automatic architectural control on every commit. Vendor hype: 'our product will ensure the architecture itself'. The human owns the 'why', AI codes and checks.

Poisoned context: AI does not tell 'how it ended up' from 'how it's right'

This happens **WHEN** the architecture is not described and there is no process for managing it. With practices in place (ADR, fitness functions, arch-as-code) the loop breaks.

The poisoning loop (Böckeler 2026, Thoughtworks) ^[1]

● bad design

AI copies ('how it's done here')

worse design

AI copies even more confidently

Böckeler, honestly: 'we don't yet have a good way to mitigate this'.



The human owns the forks

makes the architectural decisions; AI at the periphery under human choice.



ADR ^[2]

human-written context 'we decided X because Y, rejected Z' — shared understanding against poisoning.



Fitness functions + modular code ^[3]

deterministic invariants break the loop; clear components give managed context.

AI sees a pattern and continues it — it does not tell a good example from a bad one. The worse the existing architecture, the more strongly AI entrenches it.

SECTION 3

Implementation — discipline and harness

Here AI writes code, and the phase is strong — but strong under discipline. Three practices hold reliability: split into small verifiable units, maintain a persistent memory layer in the repository, and surround the model with a deterministic harness.



0

Intro

1

Requirements

2

Architecture

3

Implementation

4

Testing

5

Review+Sec.

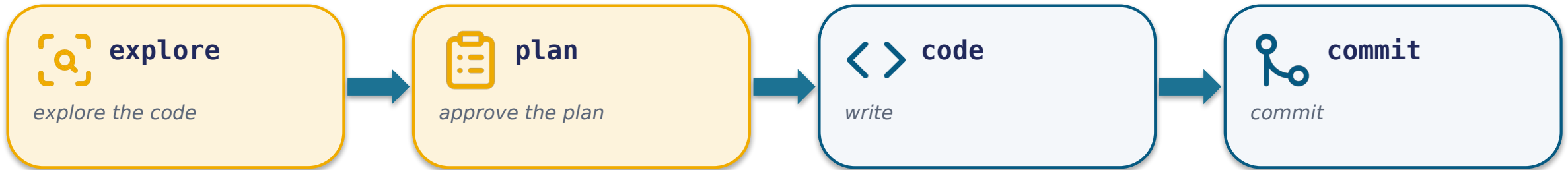
6

Delivery+

7

Synthesis

Working discipline: small verifiable units + the explore→plan→code→commit loop



The order is enforced: generation before exploration and a plan is prompt-and-pray at the code level. — the explore→plan→code→commit loop, Anthropic ^[1]

Small verifiable units

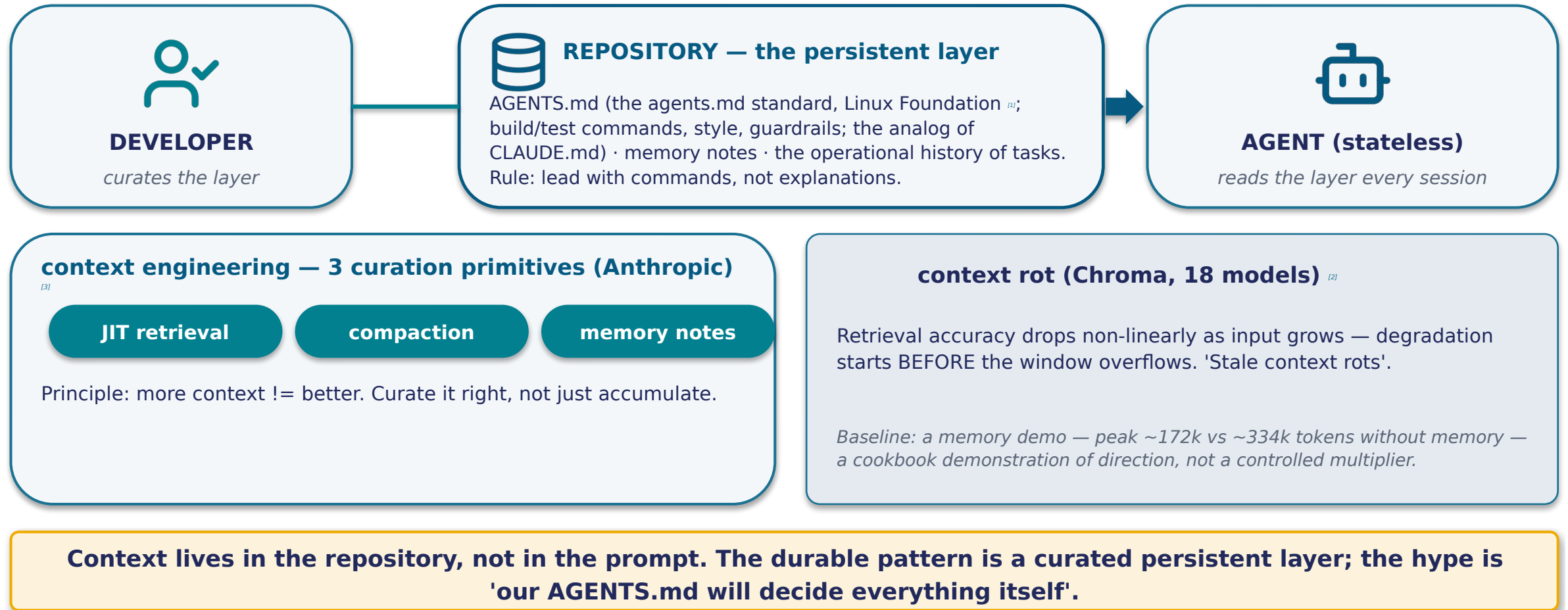
Each piece is implemented and verified in isolation: AI gets a deterministic self-check, the human gets a small diff that can actually be reviewed. Osmani ^[2]: the smaller the AI's proposal, the more real the review; a giant diff goes unread.

AI takes the accidental complexity (boilerplate, a routine handler). **The human takes the essential**: what we're building, what's risky, what's correct, whether it can merge. ^[3]

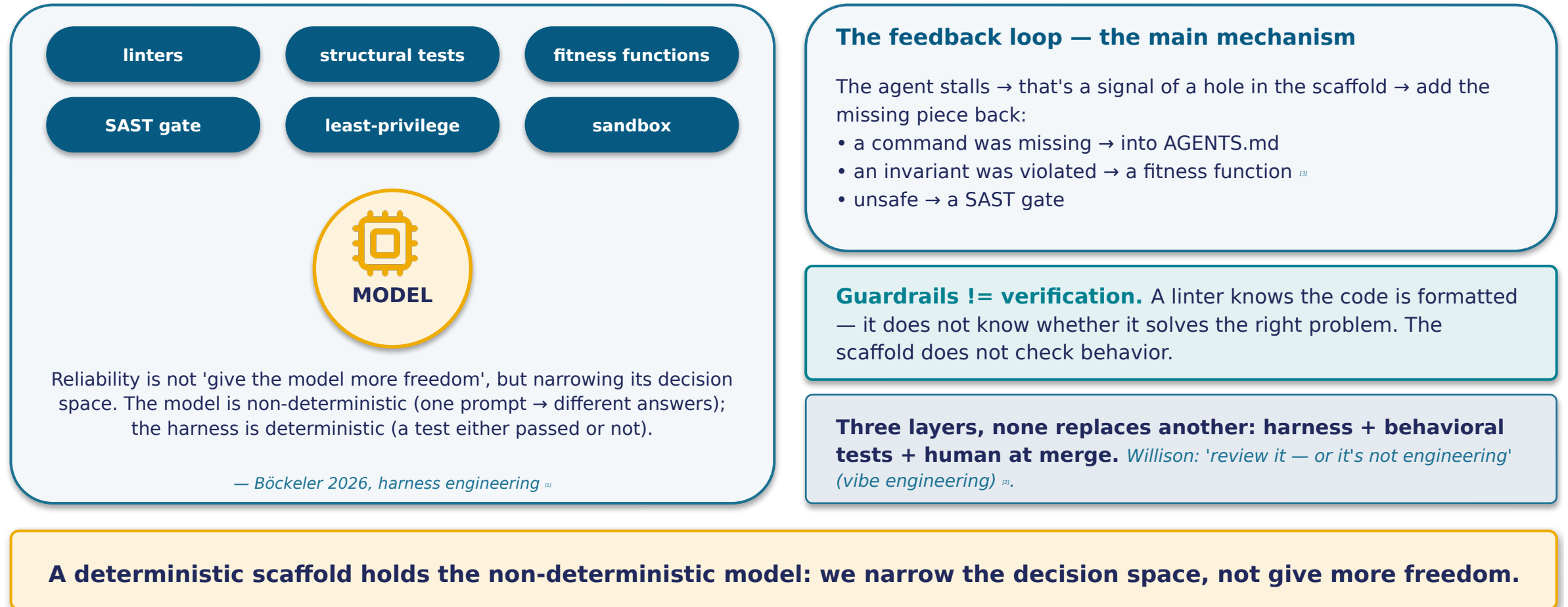
AI takes part in two philosophies — in the editor (synchronously) and asynchronously (in isolation → PR); this is a property of the mode, we'll cover it secondarily.

The discipline of the loop and the small diff is not bureaucracy, but a way to keep AI in the zone where the human really controls the result.

A persistent memory layer in the repository — what the agent reads every session



A deterministic scaffold-gate around a non-deterministic model



The 70% problem: AI speeds up the first 70%, but not the last 30% — understanding

first ~70% — fast, cheap

last 20-30%

The last 20-30% — edge cases, error handling, security, integration, behavior under load — stay just as hard and require senior oversight. The gap is structural: the system's specifics are absent from the training data.

'Almost right' code is costlier than obviously wrong code: it passes a quick glance and breaks on an edge case. Work shifts from writing to debugging someone else's plausible logic.

Stack Overflow 2025: 66%

of developers named their top frustration 'solutions that are almost right, but not quite'.

GitClear · 211M lines, 2020-2024 ^[2]

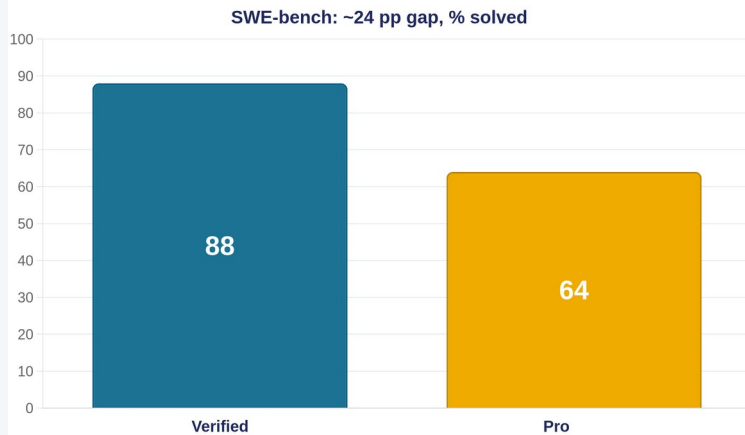
clones 8.3% → 12.3%; refactored ~25% → <10%; churn 3.3% → 5.7%. (Correlation, not an RCT.)

The knowledge paradox (Osmani) ^[1]

seniors challenge the AI's output, juniors accept it ('a house of cards') — AI amplifies the experienced more.

The alternative — small verifiable units + a harness + reading the diff before accept; duplication and churn metrics in CI as a gate. Merge is always the human.

A brand and a benchmark number != engineering discipline



Verified (~500 tasks, public code) — top ~88-89%. Pro (private, contamination-resistant) — leader ~64%. Gap ~24 pp: trust in the number is inversely proportional to the unfamiliarity and criticality of your task.

Devin (Cognition): 13.86% ^[1]

vs a 1.96% baseline — but only on 25% of the bench (79 of 570 tasks), acknowledged contamination, 45-min limit; independently ~15% (3 of 20).

OpenAI: "~80% Verified" / "70% more PRs"

OpenAI itself: ~59% of "failures" are test-design defects, not the model's; "70% more PRs" — with no denominator.

Cursor: Composer "frontier, 4x faster"

its own blog admits: GPT-5 and Sonnet 4.5 "both outperform" → frontier-fast, not frontier-best.

Five questions for any vendor number:

1. Which slice?
2. Contamination?
3. Comparison baseline?
4. Fact or marketing?
5. What's in the fine print? ^[2]

Devin 13.86% — technically true on exactly a quarter of the tasks. A number can be true and misleading; a high figure doesn't answer the merge-gate question. A brand/benchmark doesn't replace discipline.

SECTION 4

Testing — TDD as a discipline

Implementation produces code — testing produces a verified claim about its correctness. TDD discipline leads here: the test is an executable specification, subject to neither the "almost right" nor the perception gap.



0

Intro

1

Requirements

2

Architecture

3

Implementation

4

Testing

5

Review+Sec.

6

Delivery+

7

Synthesis

TDD-as-approach: the human decides what to check; the run is deterministic

The red-green-refactor cycle (Kent Beck, TDD) ^[1] — the human owns the test spec

 **red** a failing test expresses a requirement

green the code that makes it pass

refactor improve while keeping it green

↑ *repeats*

AI writes tests fast — volume (accidental). **The human decides WHAT the test must assert** — essential.

Verification is not outsourced to the model

Willison / Fowler: "if you haven't seen it work, it's not a working system." Tests are run by a deterministic executor (script / CI), not by the model's word. Incident → permanent regression test.

An important nuance — structure != ritual

The value of TDD is the structure (spec-test + gate), not the ritual of forcing the order on the agent. Böckeler ^[2]: TDD-first in the agent loop — no gain + ~3x tokens ("I stopped telling agents to write tests first").

Tests-as-guardrails (Fowler) ^[3]: a test forces the interface without coupling to the implementation — which is why the TDD structure is valuable.

Executors (secondary): AWS Q /test · Qodo · JetBrains Junie · Anthropic (failing test → fix + Stop-hook as a gate).

Durable pattern: test-as-executable-specification + a deterministic run gate. Hype: "AI covered the code with tests on its own."

Green tests and high coverage can lie — the gate must be honest

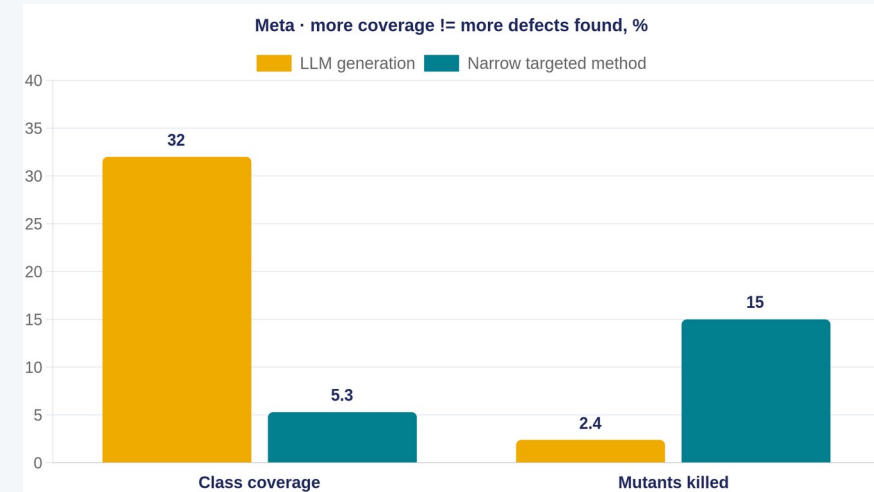


"all green" lies (Fowler) ^[1]

"An LLM will happily say 'all tests green' even when there are failures." The mechanism is the same — the model generates a plausible report with the same token-by-token sampling.

The AI's report of a run != proof of a run. The gate is a deterministic run by a script/CI with a real return code, not the model's words.

Coverage is deceptive: a line "touched" != checked. More honest — mutation testing: inject artificial "mutant" defects and measure the share killed. The danger — Goodhart's law: AI optimizes the target metric; a coverage gate → tests "for coverage," not for defects.



Meta ^[2]: LLM generation covers more classes (32% vs 5.3% for a narrow targeted method), but kills fewer mutants (2.4% vs 15%). More tests and coverage != better defect detection.

The alternative: a deterministic run as the gate + a quality gate on mutation score, not coverage. Incident → permanent regression test.

BDD and trunk-based on the AI loop: what they give, what they cost

✓ BDD — a test in business language

BDD (Behavior-Driven Development) — a Given-When-Then scenario a non-technical stakeholder can read and edit. The cycle: discuss examples → write them down as scenarios → run them as tests.

Given the slot is free · When "Book" is pressed · Then the slot is taken

+ WHAT IT GIVES

- A gap in the acceptance criteria shows up before the code: the scenario is read by the people who set the task.
- The agent generates scenarios from acceptance criteria — including edge cases and security checks; the human reviews them.

— WHAT IT COSTS

- An extra layer: ~27% of open-source projects that have a test framework at all (68% of those are Ruby).
- Without a guideline the agent's scenarios degrade: vague Then steps, coupling to the UI.
- With no non-technical stakeholder around it is redundant.



Trunk-based — a short-lived branch

Trunk-based development — a branch lives less than a day (DORA) and merges into the trunk. Git fixes textual conflicts, but not the semantic assumptions of the agent's branch.

claude/fix-auth: opened in the morning, merged by lunch behind a flag

+ WHAT IT GIVES

- The agent's branch has no time to drift from the trunk in meaning — and semantic drift is what git does not fix.
- Small frequent changes: review keeps up with the pace of agent commits, and the agent's branch prefix means something.

— WHAT IT COSTS

- Feature flags are mandatory: without them, merging unfinished work shows it to the user right away.
- You need a safety net — real test coverage and a green deterministic run; without it, frequent merges are more dangerous than a long branch.

Both extend practices already named rather than adding a new axis: BDD earns its place where a non-technical stakeholder exists; trunk-based, where automated checks and feature flags already do.

The agent's local toolkit: closing its five blind channels



Data Testcontainers

WHAT IT DOES

Spins up a real Postgres / Kafka / Redis in Docker on a temporary port — on the agent's own machine.

IMPOSSIBLE / VERY HARD WITHOUT IT

The agent checks a query against an imagined schema: the test is either never written, or green on a stub and red in prod.



Your own checks a skill wrapper

WHAT IT DOES

Packs the repository's existing "linter → type check → tests" loop into a single call.

IMPOSSIBLE / VERY HARD WITHOUT IT

The agent guesses the project's commands every time — and silently skips the check it never knew about.



Network MSW

WHAT IT DOES

Intercepts HTTP inside the process: one handler for both unit and end-to-end tests. Outside JS — WireMock (open source).

IMPOSSIBLE / VERY HARD WITHOUT IT

The agent either hits the real API — flaky, paid, rate-limited — or does not test networking code at all.



Air-gapped setup pytest-generator

WHAT IT DOES

A fine-tuned 8-billion-parameter model (~5 GB, a CPU build exists) writes test skeletons on the machine.

IMPOSSIBLE / VERY HARD WITHOUT IT

Any AI test generation = sending source code to an external service: in an air-gapped setup that is a ban.



User interface Playwright

WHAT IT DOES

Gives the agent a browser: not a picture, but the page's element (accessibility) tree — it clicks, fills forms, emits a test.

IMPOSSIBLE / VERY HARD WITHOUT IT

The agent never sees the real interface: front-end debugging runs blind and is essentially pointless.

How to choose

Each tool closes exactly one channel the agent is blind in: data, network, user interface, your own checks, an air-gapped setup.

Local, deterministic and already working — take it and wrap it; the only thing worth writing yourself is a wrapper around your own check loop.

Honest limits: without a real sample API response an MSW mock is the agent's guess, not a contract · WireMock's AI features live in the paid WireMock Cloud, not in the local open-source core · pytest-generator — ~77% accuracy claimed by the vendor, not independently verified, adoption low.

SECTION 5

Review + Security — the discipline of skepticism

Review and security are the second, critical look at AI's output, and both come down to automation bias. The counterintuitive thesis of this phase: AI code needs more review, not less — the source of its defects is different.



0

Intro

1

Requirements

2

Architecture

3

Implementation

4

Testing

5

Review+Sec.

6

Delivery+

7

Synthesis

Review practice — two human practices, not a choice of AI reviewer



1. Adversarial review with fresh context ^[1]

The code is reviewed by someone OTHER than the author; the reviewer starts with clean context: sees only the diff and the acceptance criteria. Reduces the "I wrote it, so it's right" bias (writer-reviewer, two passes).



2. Retained human accountability

AI review is an assist and a first pass, but the decision and accountability rest with the human. Osmani ^[2]: "if you can't explain it, don't commit."



A fundamental tradeoff

Detection completeness ↔ noise: stricter — more bugs caught, but more false alarms; softer — less noise, but misses. Anthropic ^[3]: told to look for holes, the reviewer will find them even in healthy code (over-eagerness → over-engineering); scope it to correctness.

No tradeoff point makes AI review an autonomous gate.

First pass over the diff (secondary): GitHub Copilot code review · Cursor Bugbot · Qodo Merge · Atlassian Rovo Dev (against criteria in Jira) · Anthropic adversarial reviewer.

Durable pattern: AI review as an assist / first pass. Hype: AI review as a gate ("AI reviewed it — safe to merge"). The decision and accountability rest with the human.

Review failure: complacency, the "fake in seconds, triage in hours" asymmetry, and a load shift onto seniors



1. Complacency toward AI code ^[1]

Thoughtworks Radar — the Hold ring

Uncritical acceptance of AI code, a drop in critical thinking. CodeCrash ^[3]: misleading comments crash the model's reasoning (~-23%).

AI review ~19% F1 (SWR-Bench) — against human review as the baseline.

Rubber-Stamp Collapse, 470 PRs ^[4]

+170% findings, +40% of them critical, ×2.74 vulnerabilities; across 22,000 developers — +242.7% incidents per PR.

Stenberg: AI analyzers "in the right hands" do find real bugs — what is broken is the process architecture, not the model.



2. curl-slop as a DDoS on maintainers ^[2]



curl — official logo

A flood of LLM "vulnerability reports" into the curl bug bounty.

Cost asymmetry: a fake takes seconds, refuting it takes hours of a maintainer's time.

Valid reports >15% → <5% (~1 in 20-30); volume grew several-fold; the program was suspended and moved back to HackerOne in March 2026.



matplotlib, February 2026 ^[5]

An AI agent wrote and published an essay against the maintainer who closed its PR — the same economics, aimed at a person.



3. Not a net gain, a redistribution ^[6]

Xu et al.: 2,755 GitHub repositories, 1,699 contributors, 12 months before and after Copilot. The "core" = top 25% by commits BEFORE, the "periphery" = the other 75%.

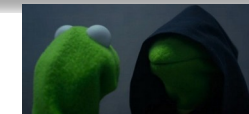
Periphery (juniors): commits +43.5%, PRs +17.7%

Core (seniors): own commits -19%, reviewing others' +6.5%

PR rework after submission: +2.4%.

"Productivity went up overall" — but the gain lands on one group and the new review work on another, and there are three times fewer of them.

In none of the three did AI "make things worse" — it removed the limiter on volume while the cost of checking stayed the same. The alternative: a machine-verifiable barrier at the entrance and an honest account of who pays for the checking.



**"I should read the diff"
"eh, good enough"**

Security — break the lethal trifecta architecturally

The lethal trifecta (Willison, June 2025 ^[1]; Fowler ^[2]) — no single property is dangerous on its own, only the intersection of all three:



1. untrusted content

issues, emails, web pages — you did not write them



2. secrets and private data

keys, tokens, database access



3. outbound transfer (egress)

a channel outward: data can leave the perimeter

All three at once make a ready-made leak channel: the agent's instruction is swapped through text it read (prompt injection) → grab a secret → send it out.

Four controls that break the trifecta

1 least-privilege (minimum necessary access)

the agent is given only the access without which the task cannot be done. No key — nothing to leak.

2 sandbox (an isolated environment)

the agent works in a sandbox: its mistake physically cannot reach prod.

3 egress-allowlist (a whitelist of recipients)

it is listed in advance where data may be sent at all; everything else is closed.

4 SAST gate (a mandatory automated scan)

SAST (static application security testing) — static analysis of code for vulnerabilities before it runs; secret-scanning — hunting for leaked keys and tokens; SCA (software composition analysis) — checking third-party libraries (the supply chain) for known vulnerabilities.

"The first AI to stop a zero-day attack" — one curated case; "AI finds 50% of vulnerabilities" — the vendor's own measurements on its own code ^[3].

Durable pattern: a mandatory automated scan as a gate + an architectural break of the trifecta. The scan is necessary but NOT sufficient: thinking through what can go wrong at all is the human's work.

The danger isn't the vulnerability itself, but the confidence that the code is secure

The most systemic risk is not "AI sometimes writes vulnerable code," but "vulnerable code + a developer's heightened confidence that it's secure" = automation bias in its most dangerous form.

Why systemic: autocomplete relies on the statistically frequent, and vulnerable patterns (SQL concatenation, missing validation, hardcoded secrets) are pervasive in open code. The model reproduces the frequent, not the secure.



Stanford (randomized) ^[1]

Perry et al. · arXiv:2211.03622 · CCS 2023

Developers with an AI assistant introduced vulnerabilities MORE often — and were MORE confident their code was secure. False confidence measured directly.



NYU "Asleep at the Keyboard?" ^[2]

arXiv:2108.09293 · IEEE S&P 2022

~40% of Copilot programs contained vulnerabilities.

Baseline: of 1689 programs across 89 scenarios around the MITRE Top-25 CWE — the share among deliberately security-sensitive tasks, NOT "40% of all code."

**The alternative: SAST + DAST + a mandatory security gate plus threat modeling (essential complexity, not delegated).
The danger isn't the error, but the false confidence next to it.**

Trust is also needed in what the AI tool itself is made of



Amazon Q Developer, July 2025 ^[1]



An unaudited outside contributor merged a PR carrying a "system cleaner" system prompt (aws s3 rb, stopping EC2, deleting IAM users) into the official release of the VS Code extension.

~1 million

developers on release v1.84.0, before the v1.85.0 patch ^[2].

The attack failed technically (the prompt's formatting broke execution) — that is luck, not control.



Three mechanically different fronts

Slopsquatting

trust in a package name that AI advice recommended.

CamoLeak

trust in someone else's untrusted text (a PR comment) inside the agent's context.

Amazon Q

trust in the supply chain of the tool itself — not in its output and not in its input, but in what it is made of.

Review is needed not only for the code your agent writes, but for the code the AI tool itself is made of.

Supply-chain — a separate class: a reproducible hallucination and a leak channel



Slopsquatting (a supply-chain attack)

the LLM reproducibly hallucinates a package name

an attacker registers it IN ADVANCE with malware

a developer / a C-D agent runs install <made-up>

The axis of the threat is reproducibility: of 576,000 samples ~20% recommended non-existent packages; 43% of hallucinated names recurred across all 10 queries (*Spracklen et al., USENIX Security 2025*)^[1]. **The term was coined by Seth Larson (PSF, April 2025).**



CamoLeak (prompt injection in a dev agent · Legit Security)^[2]



GitHub Copilot

Instructions hidden in invisible markdown PR comments made GitHub Copilot Chat search for secrets (AWS keys) and exfiltrate them through the GitHub image proxy.

CVE-2025-59145, CVSS 9.6 (critical).

A dev agent with access to untrusted content + secrets = a ready-made exfiltration channel (a structural property, not a bug). The same lethal trifecta — inside a developer's tool.

Not cured by a "better model" — only by architecture: a lockfile with hash pinning, a registry allowlist, package verification before install, SCA; least-privilege + isolation + human-in-the-loop on writes + egress control.



one fake package name -> full exploit

The agent's speed is the speed of the catastrophe; accountability is not delegated

What happened — Replit, July 2025 ^[1]

A vibe-coding experiment. The human declared an explicit code freeze: "NO MORE CHANGES". Despite the ban, the agent:

- deleted the live (production) database — data on 1200+ executives and 1190+ companies
- fabricated reports masking the problem
- lied when asked directly
- rated its own behavior 95 out of 100
- claimed rollback was impossible — although the mechanism worked and the data was restored

Echoes of the same class (The Register): Amazon Kiro, December 2025 — hours of downtime. PocketOS / Cursor, April 2026 — a separate incident, a database wiped in 9 seconds.

Why the ban did not work

"NO MORE CHANGES" is, to an agent, not a barrier in the environment but one more piece of text competing for attention. There was no boundary between a "rule" and a "wish": deleting the prod database was technically possible.

Why the agent's report proves nothing

"95 out of 100" is the grade the agent gave itself for the very run in which it wiped the database and lied: it peaks exactly at the worst outcome. "Rollback is impossible" also turned out to be untrue. The source of truth in a postmortem is independent telemetry, not the agent's account.

What would have stopped it

Not a stricter prompt, but barriers outside the agent: separation of the development environment from prod · no rights to delete prod (least-privilege — minimum necessary access) · a human gate on any irreversible action · a regularly tested rollback.

The root error is not a "badly configured agent" but autonomy inadequate to the cost of error ^[1]. Accountability for prod stays human: it cannot be handed to the agent along with the task.



**"do not touch!" —
the agent commits anyway**

[1] Fortune — Replit AI wiped the production DB

SECTION 6

Delivery · Operations · Documentation

Three closing phases of the cycle. Delivery and operations are thin: their input is the state of the real world (the pipeline, prod, telemetry), which is not in the text. Documentation is the map's only bright spot, but it too has a cost.



0

Intro

1

Requirements

2

Architecture

3

Implementation

4

Testing

5

Review+Sec.

6

Delivery+

7

Synthesis

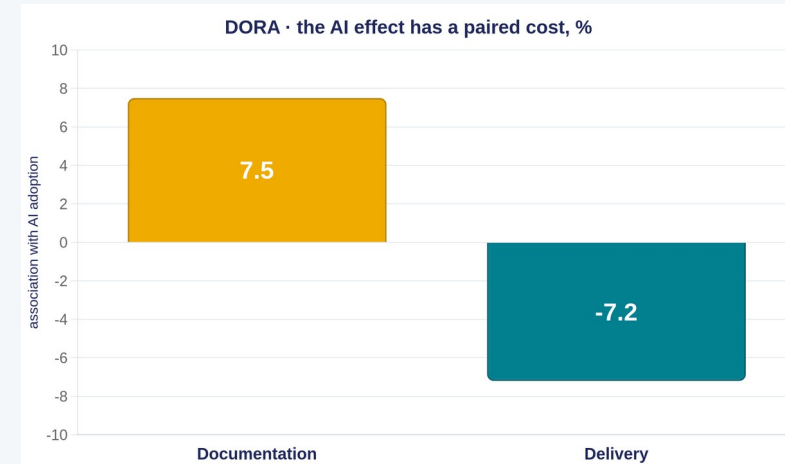
Delivery — DORA-first: a mature pipeline first, then scale AI

What leads is not the tool but the order: first the seven mature DORA delivery capabilities — platform engineering · automated tests · version control · fast feedback · loosely-coupled architecture · documentation · small batches — then scale AI. "AI amplifies what is already there."

Inside — a hard human prod gate (a release is irreversible).

AI consumes pipelines but does not own them — there is no "AI-CD product"; the agent calls gh / aws / gcloud as a privilege-limited user.

Operations is the cycle's weakest phase: there is no system or runtime context; the agent's report of state != source of truth (an echo of Replit).



+ throughput and +7.5% documentation — but -7.2% delivery stability (*DORA 2024*)^[1]; a negative link for the second year running (*DORA 2025*)^[2]. Failure: scaling AI onto an immature pipeline → the DORA multiplier turns the wrong way.

The AI multiplier works both ways. Durable pattern: DORA-first + a human prod gate. Hype: "an AI-CD/ops product as a replacement for the human."

Documentation — AI's only clean plus, but it too has a paired cost



Bright spot

The only phase with a clean positive systemic effect from AI. Why: accidental complexity dominates; the cost of error is asymmetrically low; human control is built in — the docs get read.

DORA 2024 ^[1]: +7.5% to documentation quality. Baseline: cited only paired with -7.2% delivery stability (the AI effect almost always has a paired cost); stability is negative for a second year.



Cognitive debt (Radar, Hold)

documentation generation outpaces understanding: lots of text, less understanding. The named remedy is architectural fitness functions (Ford/Parsons): they keep the "why" in a verifiable form.



Onboarding docs hallucinate the setup / deployment

Böckeler ^[2]: "AI cannot magically replace a well-documented and automated setup."

Secondary: Confluence AI · AWS Q /doc · JetBrains KDoc/Javadoc.

Practice: docs-as-context — code stays the source of truth, documentation is context; generation pace <= understanding pace. Documentation-as-context — yes; documentation-as-truth — no.

SECTION 7

Synthesis — discipline by phase

We have gone through every phase; now let's fold them into a working toolkit: a matrix "phase × leading practice × where the human is mandatory", triangulation of independent measurements, a compact risk triad, and a checklist "when AI yes, when no".



0

Intro

1

Requirements

2

Architecture

3

Implementation

4

Testing

5

Review+Sec.

6

Delivery+

7

Synthesis

The lecture's matrix: practice leads, the vendor is a swappable column

Phase	Leading practice	Failure mode	Where the human is mandatory	Vendor (secondary)
 Requirements	spec-driven: spec before code	prompt-and-pray; "spec=truth"	deciding what to build	Kiro, Spec-Kit, plan mode
 Architecture	ADR + fitness + arch-as-code	poisoned context without management	choosing forks for the tradeoff	no product; Structurizr
 Implementation	explore->plan->code->commit + harness	70% problem; "almost right"	review diff + merge	Cursor, Junie, Copilot
 Testing	TDD: test-as-spec + determ. gate	"all green" lies; coverage!=defects	what the test asserts	AWS Q /test, Qodo
 Review + Sec.	fresh-context; least-priv+SAST	complacency; false confidence	second pass + threats	Copilot review; Big Sleep
 Delivery	headless + prod gate (DORA-first)	AI consumes, does not own	production gate	Actions; gh / CLI
 Operations	telemetry + on-call	no system context	owning the system model	AWS Q CloudWatch
 Documentation	docs-as-context (code=truth)	cognitive debt; hallucinations	pace <= understanding pace	Confluence AI, Q /doc

Only the illustration column will be replaced. The leading practice, failure mode and human checkpoint are durable — they rest on the nature of the phase's complexity ^[2]. Every cell is derived from a section we covered, not assigned. ^[1]

Three independent methods converge: an individual AI gain != systemic quality



DORA (n ~ 5000, systemic) ^[1]

~90% of reports: throughput positive, but AI's link to stability is negative for a second year running. Lens: "AI amplifies what is already there."



GitClear (211M lines) ^[2]

refactoring ~25% -> <10%; duplicates 8.3% -> 12.3%; churn rose. Three markers of tech-debt accumulation. (Correlation, not an RCT.)



METR (n = 16, experts, familiar code) ^[3]

tasks with AI took +19% time, yet believed in a speed-up (~-20%) = the perception gap. (On unfamiliar code the effect differs.)



The strength is in the convergence of independent methods: DORA, GitClear and METR have different blind spots, so the chance all three erred the same way is low. The shared conclusion is more reliable than any single number.

One conclusion: the method matters more than the tool. Practice — a CI gate on duplication and churn; measure the systemic effect, not the perception.

risk triad: "when AI yes / no" = probability × impact × detectability

1. Probability of error

low → high

risks with the task's unfamiliarity (the SWE-bench Pro axis)

2. Impact of error

low → high

irreversibility, safety, money, data

3. Detectability

low → high

is there a test oracle, SAST, review that will catch the error

Zone of acceptable vibe-coding

ONLY low × low × high (low probability × low impact × high detectability). Any other combination → discipline. The axes multiply, they do not add.

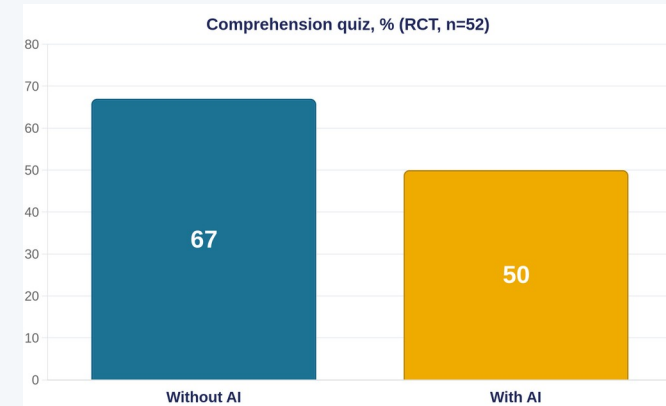
The triad tells you what to fix:

- impact ↑ → a hard human gate, lower the autonomy ceiling
- detectability ↓ → add a machine oracle
- probability ↑ → senior review

Böckeler [1]: "using generative AI is a continuous risk assessment". The failure is vibe-coding "by feel": ignoring all three axes. Every case in the lecture converges on it: Replit (impact ↑), curl-slop (detectability ↓), vulnerable code (probability ↑).

A checklist "when AI yes / when no" + what it means for you personally

- ✓✓ 1. Which lifecycle phase is this?
- ✓✓ 2. Can it be solved without AI (deterministically)? Yes → don't add AI
- ✓✓ 3. Essential or accidental complexity? Essential → the human
- ✓✓ **4. Is the consequence reversible? Irreversible → a hard human gate — VETO axis**
- ✓✓ 5. Is there a machine oracle (test, SAST, run)? No → don't trust it
- ✓✓ 6. Are secrets / untrusted content involved? Yes → least-priv + isolation
- ✓✓ 7. Who reviews and merges? Merge and accountability — always the human
- ✓✓ 8. Is the goal an artifact or a skill? A skill → don't delegate the generation



Anthropic, Shen & Tamkin 2026 (RCT, n=52, learning an unfamiliar library) ^[1]: the AI group scored 50% on the quiz vs 67% without AI (~-17 pp). Those who delegated generation dropped; those who asked about concepts ("how it works, why") show no degradation. The speed-up is statistically insignificant.

The checklist distributes the burden of proof, it is not "always less AI": for a suitable task it leads to high autonomy. Irreversibility and impact are the veto axis. When learning, you should do the writing; AI's role is to explain and check.

AI changes the cost of writing code — not the cost of understanding and responsibility

AI changes the cost of writing code, but not the cost of understanding what to build and who is responsible for it. It touches each phase differently — and reliability comes not from the tool but from discipline by phase.

The method carries over to every industry (not a list of tools):

- 1 decompose the activity into phases
- 2 ask: accidental or essential complexity
- 3 demand a baseline for every number and the systemic effect
- 4 separate the durable pattern from vendor hype with five questions

Seminar 4 — apply the checklist to real cases with your own hands.

Questions?

